

Regular Expressions

- Regular expressions describe regular languages

- Example: $(a + b \cdot c)^*$

- describes the language

$$\{a, bc\}^* = \{\lambda, a, bc, aa, abc, bca, \dots\}$$

Recursive Definition

Primitive regular expressions: $\emptyset$, λ , α

Given regular expressions r_1 and r_2

$$r_1 + r_2$$

$$r_1 \cdot r_2$$

$$r_1^*$$

$$(r_1)$$

Are regular expressions

Examples

A regular expression:

$$(a + b \cdot c)^* \cdot (c + \emptyset)$$

Not a regular expression:

$$(a + b +)$$

Example of Regular Expression and Language

$R = \epsilon \quad L(R) = \{\epsilon\}$

$R = \epsilon \quad L(R) = \{\epsilon\}$

$R = a \quad L(R) = \{a\}$

$R = a + b \quad L(R) = \{a, b\}$

$R = a.b \quad L(R) = \{ab\}$

$R = (a + ba) \quad L(R) = \{a, ba\}$

$R = (a + ba).b \quad L(R) = \{ab, bab\}$

$R = (a + ba).(b + a) \quad L(R) = \{ab, aa, bab, baa\}$



$R = (a+b)^2 \Rightarrow (a+b)(a+b) \quad L(R) = \{aa, ab, ba, bb\}$

$R = (a+b)^* \Rightarrow L(R) \{ \quad \}$

$R = (ab)^* \Rightarrow L(R) \{\text{epsilon}, ab, abab, ababab, \dots \}$

$R = (a+b)^*(a+b) \Rightarrow L(R) \{ \quad \}$



Start with ab

End with abb

Substring aba

Start and end with a

Languages of Regular Expressions

$L(r)$: language of regular expression r

- Example

$$L((a + b \cdot c)^*) = \{\lambda, a, bc, aa, abc, bca, \dots\}$$

Definition



- For primitive regular expressions:

$$L(\emptyset) = \emptyset$$

$$L(\lambda) = \{\lambda\}$$

$$L(a) = \{a\}$$

Definition (continued)

- For regular expressions r_1 and r_2
$$L(r_1 + r_2) = L(r_1) \cup L(r_2)$$
$$L(r_1 \cdot r_2) = L(r_1) L(r_2)$$
$$L(r_1^*) = (L(r_1))^*$$
$$L((r_1)) = L(r_1)$$

Example

Regular expression: $(a + b) \cdot a^*$

$$L((a + b) \cdot a^*) = L((a + b)) L(a^*)$$

$$= L(a + b) L(a^*)$$

$$= (L(a) \cup L(b)) (L(a))^*$$

$$= (\{a\} \cup \{b\}) (\{a\})^*$$

$$= \{a, b\} \{\lambda, a, aa, aaa, \dots\}$$

$$= \{a, aa, aaa, \dots, b, ba, baa, \dots\}$$

Example



- Regular expression $r = (a + b)^*(a + bb)$

$$L(r) = \{a, bb, aa, abb, ba, bbb, \dots\}$$

Example



- Regular expression $r = (aa)^*(bb)^*b$

$$L(r) = \{a^{2n}b^{2m}b : n, m \geq 0\}$$

Example



- Regular expression $r = (0 + 1)^* 00 (0 + 1)^*$

$$L(r) = \{ \text{all strings with at least two consecutive } 0 \}$$

- 
- Now consider another language L , consisting of all possible strings, defined over $\Sigma = \{a, b\}$. This language can also be expressed by the regular expression

$$(a + b)^*.$$

- Now consider another language L , of strings having exactly double a , defined over $\Sigma = \{a, b\}$, then it's regular expression may be

$$b^*aab^*$$

- 
- Now consider another language L, of even length, defined over $\Sigma = \{a, b\}$, then it's regular expression may be

$$((a+b)(a+b))^*$$

- Now consider another language L, of odd length, defined over $\Sigma = \{a, b\}$, then it's regular expression may be

$$(a+b)((a+b)(a+b))^* \text{ or } ((a+b)(a+b))^*(a+b)$$

Remark



- It may be noted that a language may be expressed by more than one regular expressions, while given a regular expression there exist a unique language generated by that regular expression.

- Example:

- Consider the language, defined over $\Sigma = \{a, b\}$ of words having at least one a , may be expressed by a regular expression $(a+b)^*a(a+b)^*$.

- Consider the language, defined over $\Sigma = \{a, b\}$ of words having at least one a and one b , may be expressed by a regular expression

$$(a+b)^*a(a+b)^*b(a+b)^* + (a+b)^*b(a+b)^*a(a+b)^*.$$

- 
- Consider the language, defined over $\Sigma = \{a, b\}$, of words starting with double a and ending in double b then its regular expression may be $aa(a+b)^*bb$
 - Consider the language, defined over $\Sigma = \{a, b\}$ of words starting with a and ending in b OR starting with b and ending in a, then its regular expression may be $a(a+b)^*b + b(a+b)^*a$

TASK

- Consider the language, defined over $\Sigma=\{a, b\}$ of **words beginning with a**, then its regular expression may be
- Consider the language, defined over $\Sigma=\{a, b\}$ of **words beginning and ending in same letter**, then its regular expression may be

TASK

- Consider the language, defined over $\Sigma=\{a, b\}$ of **words ending in b**, then its regular expression may be
- Consider the language, defined over $\Sigma=\{a, b\}$ of **words not ending in a**, then its regular expression may be . It is to be noted that this language may also be expressed by

Regular expression in real life scenario



Regular Expression

- A regular expression is a sequence of characters that specifies a search pattern in text. Usually such patterns are used by string-searching algorithms for "find" or "find and replace" operations on strings, or for input validation

Equivalent Regular Expressions

- Definition:
- Regular expressions r_1 and r_2
- are **equivalent** if $L(r_1) = L(r_2)$

Example

- $L = \{ \text{all strings without two consecutive 0} \}$

$$r_1 = (1 + 01)^* (0 + \lambda)$$

$$r_2 = (1^* 0 1 1^*)^* (0 + \lambda) + 1^* (0 + \lambda)$$

$$L(r_1) = L(r_2) = L \quad \longrightarrow \quad r_1 \text{ and } r_2$$

are equivalent
regular expr.

Task

- Q)
- 1) Let $S = \{ab, bb\}$ and $T = \{ab, bb, bbbb\}$ Show that $S^* = T^*$ [Hint $S^* \subseteq T^*$ and $T^* \subseteq S^*$]
- 2) Let $S = \{ab, bb\}$ and $T = \{ab, bb, bbb\}$ Show that $S^* \neq T^*$ But $S^* \subset T^*$

- 
- 3) Let $S = \{a, bb, bab, abaab\}$ be a set of strings. Are $abbabaabab$ and $baabbbabbaabb$ in S^* ? Does any word in S^* have odd number of b's?

Solution: since $abbabaabab$ can be grouped as $(a)(bb)(abaab)ab$, which shows that the last member of the group does not belong to S , so $abbabaabab$ is not in S^* , while $baabbbabbaabb$ can not be grouped as members of S , hence $baabbbabbaabb$ is not in S^* . Since each string in S has even number of b's so there is no possibility of any string with odd number of b's to be in S^* .

Task



Q1) Is there any case when S^+ contains Λ ? If yes then justify your answer.



Q2) Prove that for any set of strings S

i. $(S^+)^* = (S^*)^*$

Solution: In general Λ is not in S^+ , while Λ does belong to S^* . Obviously Λ will now be in $(S^+)^*$, while $(S^*)^*$ and S^* generate the same set of strings. Hence $(S^+)^* = (S^*)^*$.

Q2) continued...

ii) $(S^+)^+ = S^+$

Solution: since S^+ generates all possible strings that can be obtained by concatenating the strings of S , so $(S^+)^+$ generates all possible strings that can be obtained by concatenating the strings of S^+ , will not generate any new string.

Hence $(S^+)^+ = S^+$

Q2) continued...

iii) Is $(S^*)^+ = (S^+)^*$

Solution: since Λ belongs to S^* , so Λ will belong to $(S^*)^+$ as member of S^* . Moreover Λ may not belong to S^+ , in general, while Λ will automatically belong to $(S^+)^*$.

Hence $(S^*)^+ = (S^+)^*$

Regular Expression

- As discussed earlier that a^* generates $\Lambda, a, aa, aaa, \dots$ and a^+ generates $a, aa, aaa, aaaa, \dots$, so the language $L_1 = \{\Lambda, a, aa, aaa, \dots\}$ and $L_2 = \{a, aa, aaa, aaaa, \dots\}$ can simply be expressed by a^* and a^+ , respectively.
 a^* and a^+ are called the regular expressions (RE) for L_1 and L_2 respectively.
Note: a^+, aa^* and a^*a generate L_2 .

Recursive definition of Regular Expression(RE)

Step 1: Every letter of Σ including Λ is a regular expression.

Step 2: If r_1 and r_2 are regular expressions then

1. (r_1)

2. $r_1 r_2$

3. $r_1 + r_2$ and

4. r_1^*
are also regular expressions.

Step 3: Nothing else is a regular expression.

Defining Languages (continued)...

- **Method 3 (Regular Expressions)**

- Consider the language $L = \{\Lambda, x, xx, xxx, \dots\}$ of strings, defined over $\Sigma = \{x\}$.

We can write this language as the Kleene star closure of alphabet Σ or $L = \Sigma^* = \{x\}^*$

this language can also be expressed by the regular expression x^* .

- Similarly the language $L = \{x, xx, xxx, \dots\}$, defined over $\Sigma = \{x\}$, can be expressed by the regular expression x^+ .

More Practical RE Examples

$[abcd]$ means $(a \mid b \mid c \mid d)$, $[b-g]$ means $[bcdefg]$, $[b-gM-Qkr]$ means $[bcdefgMNO PQkr]$, $M?$ means $(M \mid \epsilon)$, and M^+ means $(M \cdot M^*)$

a	An ordinary character stands for itself.
ϵ	The empty string.
	Another way to write the empty string.
$M \mid N$	Alternation, choosing from M or N .
$M \cdot N$	Concatenation, an M followed by an N .
MN	Another way to write concatenation.
M^*	Repetition (zero or more times).
M^+	Repetition, one or more times.
$M?$	Optional, zero or one occurrence of M .
$[a-zA-Z]$	Character set alternation.
$.$	A period stands for any single character except newline.
$"a.+*"$	Quotation, a string in quotes stands for itself literally.

Algebraic Properties

$r \mid s = s \mid r$	$\mid$ is commutative
$r \mid (s \mid t) = (r \mid s) \mid t$	$\mid$ is associative
$r(st) = (rs)t$	concatenation is associative
$r(s \mid t) = rs \mid rt$ $(s \mid t)r = sr \mid tr$	concatenation distributes over $\mid$
$\epsilon r = r$ $r \epsilon = r$	ϵ is the identity for concatenation
$r^* = (r \mid \epsilon)^*$	ϵ is contained in *
$r^{**} = r^*$	* is idempotent

Using R.E for Tokenization

identifiers

$letter \rightarrow (a \mid b \mid c \mid \dots \mid z \mid A \mid B \mid C \mid \dots \mid Z)$

$digit \rightarrow (0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)$

$id \rightarrow letter (letter \mid digit)^*$

numbers

$integer \rightarrow (+ \mid - \mid \epsilon) (0 \mid (1 \mid 2 \mid 3 \mid \dots \mid 9) digit^*)$

$decimal \rightarrow integer . (digit)^*$

$real \rightarrow (integer \mid decimal) E (+ \mid -) digit^*$

$complex \rightarrow '(real , real)'$

R.E. and Tokens – Our job

Assign a token type to a subset of input stream, based on its matching with a RE

if	IF
[a-z] [a-z0-9] *	ID
[0-9] +	NUM
([0-9] + "." [0-9] *) ([0-9] * "." [0-9] +)	REAL
("--" [a-z] * "\n") (" " "\n" "\t") +	<i>no token, just white space</i>
.	<i>error</i>

- The fifth line recognizes comments or white space but does not report back to the parser.
- White space is discarded and the lexer resumed.
- The comments for this lexer begin with two dashes, contain only alphabetic characters, and end with newline.

SummingUP Lecture 2

RE, Recursive definition of RE, defining languages by RE, $\{x\}^*$, $\{x\}^+$, $\{a+b\}^*$, Language of strings having **exactly one aa**, Language of strings of **even length**, Language of strings of **odd length**, RE defines unique language (as Remark), Language of strings having **at least one a**, Language of strings having **at least one a and one b**, Language of strings **starting with aa and ending in bb**, Language of strings **starting with and ending in different letters**.

Example



- Regular expression $r = (1 + 01)^* (0 + \lambda)$

$$L(r) = \{ \text{all strings without two consecutive 0} \}$$